

HOPHY: A Hierarchical Hypergraph Representation for Off-Road Path and Mission Planning

Pranay Meshram¹, Charuvahan Adhivarahan¹, Prithvi Poddar², Ehsan Tarkesh Esfahani², Chen Wang¹, Souma Chowdhury², and Karthik Dantu¹

Abstract—Mission-level autonomy for disaster response, search and rescue, and tactical UGV operations requires repeated path and mission planning as terrain conditions, agent types, and objectives change. Pixel-grid search is costly for repeated kilometer-scale queries, while semantic abstractions must maintain valid costs and connectivity as conditions change. We present HOPHY (Hierarchical Off-Road Planning using Hypergraphs), a reusable hierarchical terrain representation that organizes map-scale terrain into Geometrically connected Semantic regions (GSNodes), connectivity-preserving Coarse Regions, and typed hyperedges for terrain, agent, and weather context. Hyperedge intersections select affected regions and incident edges for state updates without rebuilding the hierarchy. Across real off-road maps spanning kilometer-scale areas, HOPHY achieves 100% planning success and $< 0.01\%$ median cost deviation from the oracle (pixel A*), with substantially lower query and replanning latency than the evaluated pixel and abstraction baselines. Applied to a multi-robot task-allocation (MRTA) problem, these gains reduce total computation by $79\times$ over pixel A* and $7.2\times$ over the fastest abstraction baseline, with mission makespan comparable to pixel A*. Finally, we demonstrate HOPHY on a physical Clearpath Jackal that successfully executes a 1.5-km, eight-task mission across mixed-surface outdoor terrain and a blockage-triggered replanned route.

I. INTRODUCTION

Military logistics and search-and-rescue missions require ground robots to reach dispersed locations across large, unstructured landscapes, often where roads are unavailable or access has been disrupted [1], [2]. Delivering supplies across mountainous terrain or reaching search sites after a disaster requires routes that account for vegetation, surface conditions, and slope, as well as the capabilities of each vehicle. Coordinating a team adds another demand: deciding which robot should visit each location, and in what order, requires repeatedly estimating the cost of travel between candidate tasks [3], [4]. These estimates must remain useful as rain changes traction, a route becomes blocked, or task assignments change. Such missions therefore need planning that combines terrain-aware route quality with fast, repeated queries over kilometer-scale areas and efficient updates as operating conditions evolve.

Much of the information needed to support this planning is available before deployment. Public landcover products,

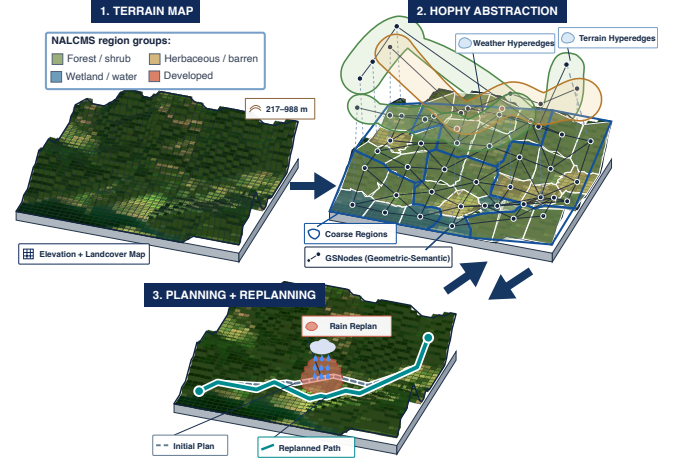


Fig. 1: HOPHY on a Mount Rainier terrain subregion. Elevation and landcover are decomposed into GSNodes, Coarse Regions, and typed hyperedges for terrain-aware planning and localized replanning. Elevation vertically exaggerated.

such as the North American Land Change Monitoring System (NALCMS) [5], identify surface classes, while digital elevation models, such as NASADEM [6], provide terrain geometry from which slope can be derived. Public weather observations and forecasts can supply additional context about precipitation and the areas it affects. Agent models complement these environmental layers by specifying mobility limits and relating terrain and operating conditions to traversal cost, travel time, or energy use. For example, the same rain-affected slope may remain traversable for one vehicle while becoming costly or infeasible for another. Translating these inputs into planning decisions requires a representation that preserves terrain connectivity and associates environmental changes with the regions and agents to which they apply.

The challenge is to make this representation efficient across an entire mission. Pixel-level search retains terrain detail, but repeatedly searching a large raster and evaluating traversal costs can dominate task-allocation and replanning time. Further, restricting repeated long-range searches within the abstraction and efficiently identifying which planning states must change under overlapping terrain, weather, and agent constraints is another challenge. A hierarchy can reduce the search space, while an explicit association between context and planning state can support targeted updates.

We present Hierarchical Off-Road Planning using Hypergraphs (HOPHY), a reusable terrain representation that

¹Department of Computer Science and Engineering, University at Buffalo, NY 14260, USA. {pranaywa, charuvah, cwx, kdantu}@buffalo.edu

²Department of Mechanical and Aerospace Engineering, University at Buffalo, NY 14260, USA. {prithvid, ehsanesf, soumacho}@buffalo.edu

combines these capabilities. HOPHY organizes landcover and elevation into Geometrically connected Semantic regions (GSNodes) and connectivity-preserving Coarse Regions, supporting coarse-to-fine routing followed by pixel-level refinement. Typed hyperedges encode overlapping terrain, agent-feasibility, and weather memberships over the same regions. Their intersections select the regions and incident connections whose costs, feasibility, or attributes must change, while retaining the underlying hierarchy for subsequent queries. This coupling of hierarchical search and contextual updates enables repeated path-cost estimation within mission optimization. Figure 1 illustrates the representation. Our contributions are:

- **A reusable hierarchical terrain abstraction** for repeated off-road path queries, combining connected semantic regions, coarse routing, and pixel-level refinement. Across the evaluated maps, median query times are 28–630 ms, with 100% planning success and less than 0.01% median cost deviation from pixel A*.
- **A typed hyperedge update mechanism** that maps combinations of terrain, agent, and weather context to affected regions and connections. It supports rain and blockage updates without rebuilding the hierarchy, with mean update-plus-query latency below 400 ms across the evaluated replanning conditions.
- **Validation from path queries to mission execution** on real terrain maps spanning 9–100 km², including multi-agent task allocation and a physical ground robot demonstration. In the five-agent, fifty-task experiment, HOPHY reduces planning computation by 79× relative to pixel A* and 7.2× relative to CLEAR [7], with estimated mission makespan comparable to pixel A*. A Clearpath Jackal executes a 1.5-km, eight-task mission and a blockage-triggered replacement route.

II. RELATED WORK

Terrain abstractions and hierarchical planning. Off-road planning representations progressively reduce the search space from pixel grids to structured abstractions. Fixed-cluster abstractions [8] accelerate binary-grid search, but changing off-road costs invalidate cached connections. Flat semantic-geometric graphs [7] built from landcover and elevation substantially accelerate path queries and provide the closest flat-abstraction baseline in our evaluation. Hierarchical approaches [9], [10] further stratify search through coarse-to-fine routing and risk-aware traversal over unstructured terrain. Context changes involving agents, weather, or blockages still require identifying and updating affected costs and connections. HOPHY maintains typed memberships for this identification alongside its reusable hierarchy.

Dynamic replanning and traversal costs. Two lines of work address cost changes after initial planning but in orthogonal ways. Incremental search methods [11], [12] repair shortest-path state when changed edges are supplied. Learned cost models [13], [14] estimate surface-specific traversal costs from sensor observations, producing signals that a planner can consume. Repair methods [11], [12] take changed

edges as input, while cost models [13], [14] produce per-surface estimates but do not organize the planning space. The proposed abstraction connects these roles: hyperedge intersections identify the affected planning edges and expose their updates to downstream search.

Hypergraphs for repeated mission-planning queries. Hypergraphs have encoded natural-terrain relationships [15] and coupled constraints in multi-robot task and motion planning [16], exploiting hyperedges as constraint containers that group entities sharing a relationship. Their set-intersection semantics make them equally suited as overlap indices: given overlapping terrain, agent, and weather memberships, their intersection directly identifies the affected planning subset without scanning every terrain node. HOPHY uses these indices to maintain contextual state across repeated planning calls. MRTA solvers [3] and course-of-action generators [4] expose the downstream demand for this capability, as both require repeated path-cost evaluations whose latency determines whether mission-level optimization is tractable at planning time. The proposed representation makes this inner loop fast enough to serve those repeated evaluations.

III. HIERARCHICAL REPRESENTATION WITH HYPERGRAPHS

HOPHY is constructed offline in two stages, shown in Figure 2 and summarized in Algorithm 1. First, terrain maps are converted into a spatial hierarchy of GSNodes and Coarse Regions, with adjacency and bridge edges preserving connectivity. Second, typed hyperedges associate these GSNodes with terrain properties, agent feasibility constraints, and weather conditions. The following subsections describe these stages in order. The resulting representation supports the online path queries and contextual updates described in Section IV.

A. Hierarchical Representation Building

Terrain preprocessing: At Level 1 (Raster Map) in Figure 2, cells are labeled by landcover and local grade to identify similar patches. COMBINETERRAIN forms joint class labels from landcover classes and elevation-derived grade bins. Let $\mathcal{L}$ be the landcover classes and $\mathcal{B}_E$ the grade bins computed from elevation E . Each cell x receives a label encoding the pair $(\ell(x), b_E(x)) \in \mathcal{L} \times \mathcal{B}_E$. Cells sharing a joint label therefore have the same landcover class and approximately similar local grade. In our implementation, eight landcover classes are combined with three grade bins, $[0, 10]\%$, $(10, 20]\%$, and $(20, 35]\%$, using

$$M(x) = 3(\ell(x) - 1) + b_E(x) + 1,$$

where $\ell(x) \in \{1, \dots, 8\}$ and $b_E(x) \in \{0, 1, 2\}$. This produces 24 joint labels, with a separate label 25 for all terrain above 35% grade. These labels guide segmentation by landcover and local terrain geometry, rather than by absolute elevation.

Morphological opening removes small isolated patches, and closing fills small gaps (MORPHSIMPLIFY). Applying these operations at scale α_f produces M_f ,

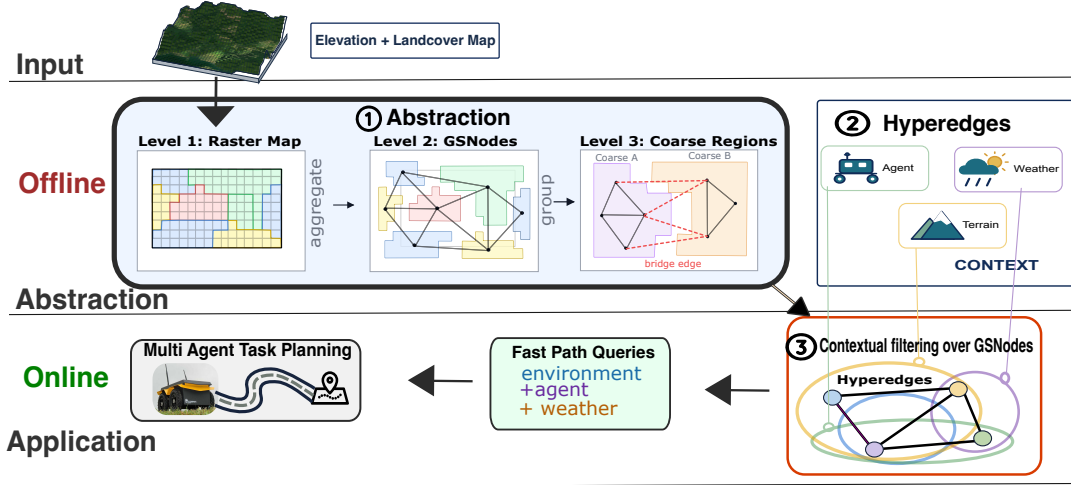


Fig. 2: HOPHY terrain hierarchy and query pipeline. Terrain inputs are decomposed into GSNodes and Coarse Regions. Typed hyperedges index overlapping contextual memberships for localized updates, repeated path queries, and multi-agent planning.

whose connected patches define the fine-segment set $\mathcal{S}$ (CONNECTEDSEGMENTS). Fine segments serve only as intermediate construction units.

Geometrically connected Semantic regions (GSNodes): Repeated pixel-level search is expensive in unstructured terrain. A GSNode provides a larger planning unit by merging connected fine segments with the same joint landcover-grade label from $\mathcal{L} \times \mathcal{B}_E$. The Level 2 graphic in Figure 2 illustrates this grouping of fine segments into GSNodes. For a connected group $B \subseteq \mathcal{S}$ of fine segments sharing a joint label, the resulting node is

$$D_B = \bigcup_{s \in B} D_s, \quad g_B = (D_B, m_B, \bar{z}_B),$$

where D_s is a segment’s geometry, m_B is the shared joint label, and $\bar{z}_B$ is the area-weighted mean elevation. Thus, each GSNode is connected and has a common landcover class and grade bin, while its shape may be nonconvex. Let $\mathcal{G}$ denote the complete GSNode set.

Coarse Regions: The GSNode graph can still contain tens of thousands of nodes. Coarse Regions restrict long-range search to a coarse corridor, with bridge edges preserving connectivity between neighboring regions (Level 3 in Figure 2).

Further simplification at scale α_c produces M_c , whose boundaries define coarse polygons $\mathcal{P}$ after merging small polygons using threshold $A_{\min}$ (COARSEPOLYGONS). During construction, fine segments are assigned to each polygon $P \in \mathcal{P}$ by overlap or centroid containment, with each segment assigned once (ASSIGNEDSEGMENTS). Segments with matching joint labels are merged within that polygon (MERGEBYTERRAIN), producing $\mathcal{G}_P$. This ensures that each GSNode has a unique coarse parent. A nonempty polygon and its nodes define a Coarse Region (CREATECOARSEREGION) as

$$c_P = (P, \mathcal{G}_P), \quad \mathcal{C} = \{c_P : P \in \mathcal{P}, \mathcal{G}_P \neq \emptyset\}.$$

The parent mapping $\pi_{G \rightarrow C}(g) = c_P$ for $g \in \mathcal{G}_P$ assigns each GSNode to exactly one Coarse Region, and every retained

Algorithm 1 Build HOPHY Hierarchy

Require: Maps L, E , scales α_f, α_c , area threshold $A_{\min}$, agent constraints $\mathcal{F}$, weather q_0
Ensure: Graphs G_G, G_C and typed hyperedges $\mathcal{E}_H$
1: Form joint terrain labels M from L and E .
2: Simplify M at α_f , then α_c , yielding M_f, M_c .
3: Extract fine segments $\mathcal{S}$ from M_f and coarse polygons $\mathcal{P}$ from M_c using $A_{\min}$.
4: Assign each fine segment to one coarse polygon.
5: Within each polygon, merge connected segments with the same joint label into GSNodes $\mathcal{G}_P$.
6: Form $\mathcal{G} = \bigcup_P \mathcal{G}_P$ and nonempty Coarse Regions $\mathcal{C} = \{(P, \mathcal{G}_P) : \mathcal{G}_P \neq \emptyset\}$.
7: Build GSNode adjacency G_G and coarse bridge graph G_C .
8: Index terrain, agent ($\mathcal{F}$), and weather (q_0) memberships as $\mathcal{E}_H$.
9: **return** $G_G, G_C, \mathcal{E}_H$

region is nonempty.

Adjacency across abstractions: GSNode adjacency defines the $GSGraph$, represented by the directed graph $G_G = (\mathcal{G}, E_G)$, where $(g_i, g_j) \in E_G$ if distinct GSNodes g_i and g_j share a boundary. Both directions are included, while their runtime costs may differ. Contraction under $\pi_{G \rightarrow C}$ yields $G_C = (\mathcal{C}, E_C)$, where

$$(c_k, c_\ell) \in E_C \iff c_k \neq c_\ell \wedge \exists (g_i, g_j) \in E_G : \pi_{G \rightarrow C}(g_i) = c_k \wedge \pi_{G \rightarrow C}(g_j) = c_\ell.$$

Offline construction. Algorithm 1 implements these definitions in one offline pass. BUILDGSADJACENCY connects neighboring GSNodes to form G_G . BUILDBRIDGEGRAPH links their parent coarse regions across boundary edges to form G_C . Finally, INDEXSETS groups GSNodes by terrain, agent feasibility, and weather context into the hyperedges defined next. The hierarchy is retained for online queries and updates.

B. Hyperedge Construction

The Hyperedges block in Figure 2 associates GSNodes with overlapping terrain, agent-feasibility, and weather memberships, without requiring spatial connectivity.

The typed hypergraph is $\mathcal{H} = (\mathcal{V}, \mathcal{E}_H, \lambda, \mathbf{X})$, where $\mathcal{V} = \mathcal{G}$, $\mathcal{E}_H \subseteq 2^{\mathcal{V}}$ is the set of hyperedges, $\lambda : \mathcal{E}_H \rightarrow \mathcal{T}$ assigns

each hyperedge a contextual type, and $\mathbf{X} : \mathcal{V} \rightarrow \mathcal{A}$ maps GSNodes to attributes. Membership $v \in e$ defines incidence. Hyperedge intersections select nodes for contextual updates, while E_G retains spatial adjacency.

We construct three membership families over the same GSNodes:

Terrain-based hyperedges ($e_{\text{landcover}}, e_{\text{grade}}$) group GSNodes by landcover class or the grade intervals defined above.

Agent-based hyperedges (e_{agent}) collect GSNodes satisfying each agent’s feasibility limits, such as the 35% grade limit for wheeled vehicles. Built after hierarchy construction, these memberships are selected for the querying agent online. Traversal costs come from a separate function (Section IV-A).

Weather-effect hyperedges (e_{weather}) collect GSNodes exposed to conditions such as rain, identifying where context-dependent traversal costs change.

Contextual updates through set operations. The Contextual filtering block in Figure 2 selects nodes for a typed update operator $\Omega = (\tau, \{e_i\}_{i=1}^k, \phi_V, \phi_E)$, where τ declares the update type (cost, feasibility, or attribute relabel), the $e_i \in \mathcal{E}_H$ are applicability memberships, and ϕ_V and ϕ_E are node- and edge-state transformations. Its support is

$$U(\Omega) = \bigcap_{i=1}^k e_i \quad \text{e.g.,} \quad e_{\text{agent}} \cap e_{\text{weather}} \cap e_{\text{landcover}} \cap e_{\text{grade}},$$

and only GSNodes in $U(\Omega)$ and their incident directed edges are transformed. Operators on disjoint attributes commute. When operators target the same attribute, a declared precedence applies (feasibility masking before cost modification). Memberships are stored as hash sets. With $E(U) = \{(u, v) \in E_G : u \in U \vee v \in U\}$, the set-selection and state-mutation cost is $O(\sum_i |e_i| + |U| + |E(U)|)$ rather than a full $O(|\mathcal{G}| + |E_G|)$ scan. This becomes a full-graph operation when every GSNode is matched. Cache invalidation is described in Section IV-A.

IV. PATH AND MISSION PLANNING

The constructed hierarchy supports rapid online route planning. We first explain how coarse-to-fine path planning uses this hierarchy under agent and environmental constraints, as summarized in Algorithm 2. We then use this approach to estimate inter-task travel costs and solve a multi-robot task allocation (MRTA) problem for a heterogeneous team.

A. Path Planning with HOPHY

Algorithm 2 implements the Fast Path Queries block in Figure 2. It maps endpoints to valid GSNodes and parent Coarse Regions, preferring representatives in the endpoints’ raster-connected components. After contextual filtering, one connected GSNode search follows a coarse corridor and widens it if needed. Coarse routes and search subsets are cached. Directed pixel-level A* then connects the original endpoints within the seeded corridor under the current mask and online traversal costs.

Algorithm 2 Coarse-to-Fine HOPHY Query

Require: Start x_s , goal x_g , graphs G_G, G_C , context q , cache $\mathcal{K}$

Ensure: Feasible flag, path π , cost J

```

1: // Map query endpoints to the maintained hierarchy
2:  $(g_s, c_s) \leftarrow \text{LOCATEGSNODE}(x_s)$ 
3:  $(g_g, c_g) \leftarrow \text{LOCATEGSNODE}(x_g)$ 
4: // Apply weather, blockage, and agent constraints
5:  $G_G^q \leftarrow \text{APPLYHYPEREDGEFILTER}(G_G, q)$ 
6: if  $g_s = \emptyset$  or  $g_g = \emptyset$  then
7:   return false,  $\emptyset$ ,  $\infty$ 
8: end if
9: // Use local search when no coarse transition is needed
10: if  $c_s = c_g$  then
11:    $\sigma \leftarrow \text{GSNODEASTAR}(G_G^q, g_s, g_g, q)$ 
12: else
13:   // Search a connected corridor across coarse boundaries
14:    $\rho \leftarrow \text{COASEROUTE}(G_C, c_s, c_g, \mathcal{K})$ 
15:    $\sigma \leftarrow \text{CORRIDORGSNODESEARCH}(G_G^q, g_s, g_g, \rho, \mathcal{K})$ 
16:   {Widen to neighboring coarse regions if needed}
17: end if
18: if  $\sigma = \emptyset$  then
19:   return false,  $\emptyset$ ,  $\infty$ 
20: end if
21:  $C_q \leftarrow \text{RASTERCORRIDOR}(\sigma, x_s, x_g, q)$ 
22:  $(\pi, J) \leftarrow \text{DIRECTEDASTAR}(C_q, x_s, x_g, q)$ 
23: return  $(\pi \neq \emptyset)$ ,  $\pi$ ,  $J$ 

```

Partial blockages and cache invalidation. Blockages mask affected pixels and exclude GSNodes exceeding the configured blocked-pixel count and fraction thresholds. Endpoint GSNodes remain candidates, raster refinement enforces partial blockages and pixel connectivity. Rain updates change friction-dependent features and clear dynamic transition, abstract-edge, and compiled inference caches, retaining the hierarchy and static descriptors. Query-index refreshes clear query-result and GSNode-subproblem caches and, when requested, route caches. Invalidation is not limited to affected edges.

Traversal Cost Interface: A representation-agnostic function supplies directed weights $w(u, v \mid q)$ from transition descriptors and agent–environment context. Signed grade and terrain transitions generally make $w(u, v \mid q) \neq w(v, u \mid q)$, so one scalar cost raster cannot represent the objective. Full-grid A* and HOPHY evaluate the same function online, with HOPHY restricting the evaluation region.

B. Mission Planning

For the Multi Task Planning block in Figure 2, we use HOPHY inside a heterogeneous Min-Max mTSP formulation [17] following the ST-SR MRTA taxonomy [3]. The objective is to minimize the latest agent completion time, $\min_j \max T_{a_j}$, over tasks in a large off-road environment. We solve the allocation with a genetic algorithm [4]. HOPHY supplies the repeated inter-task travel-time and energy estimates.

V. EVALUATION

We evaluate the proposed representation across increasingly demanding settings: repeated long-range path queries, replanning after terrain changes, mechanism and scaling analysis, multi-agent mission planning, and physical Jackal execution. This progression is organized around five ques-

TABLE I: Abstraction hyperparameters.

Map	A_{fine}	α_f	α_c	A_{min}
Wharton	0	0	4	100
Humphrey	4	2	16	500
Rainier	4	4	16	1000

Q1: Path-planning performance: Does HOPHY accelerate repeated queries without sacrificing success or path quality?

Q2: Replanning: Does this advantage persist after terrain costs or feasibility change?

Q3: Mechanisms: Which components produce the gains, and how do hyperedge updates scale?

Q4: Mission planning: Do faster queries improve a multi-agent optimization loop?

Q5: Deployment: Can the resulting paths be executed on a physical robot?

A. Experimental Protocol

Platform and maps. All computational experiments use Ubuntu 20.04 on a 12th Gen Intel Core i9-12900K with 32 GB RAM. We evaluate Wharton (W, 9 km², 355 × 268), Humphrey (H, 50 km², 1707 × 1670), and Rainier (R, 100 km², 3198 × 2613). Each map combines NASA DEM [6] elevation and NALCMS [5] landcover with a nominal source resolution of 30 m. We generate 90 distance-stratified start–goal pairs per map (270 total).

Hierarchy parameters. Table I lists the morphological ablation settings for Algorithm 1. Scales are in pixels and areas in pixel². A zero α_f disables fine smoothing. A_{fine} filters fine regions, and A_{min} controls coarse-polygon merging. The verified path-query results instead use a direct-raster hierarchy that bypasses fine-region construction.

Query and cost settings. Verified queries start in dry weather (q_0), with target speed 1.0 m/s and maximum grade 35%. Cost, risk, time, and step weights are 8, 4, 0.02, and 1. Pixel refinement uses one corridor of radius 192 pixels. Compilation precedes timing, with no graph-query warmups. The rain-replanning runner excludes GSNodes with at least 4 blocked pixels and blocked fraction 0.08, uses zero blockage-expansion hops, and retains endpoint GSNodes for raster refinement.

Objective and baselines. All planners use identical terrain inputs and a shared learned Jackal traversal objective. Taking inspiration from TERP [18], we train an MLP with two shared tanh layers (10 and 48 neurons) that predicts traversal cost from grade, roughness, friction, target speed, and distance, trained on 18,090 augmented Jackal traversal samples from BeamNG.tech, a soft-body vehicle simulation platform [19]. All methods use this same cost function to score transitions. Baselines are 2-stage PRM+A* [9], CLEAR [7], RRT*, and RRT-Connect.

Reporting. Because a single scalar measure can obscure complementary failure modes in geometric evaluation [20], we report success, cost deviation, length deviation, and query time separately rather than collapsing them into one score. All methods were evaluated on the same hardware using identical start–goal pairs and a shared traversal-cost model. Query time includes required search-time model evaluations. RRT* and RRT-Connect use 3000 and 10000 iterations, respectively, with three attempts. Cost and length use the

TABLE II: Initialization time (s). HOPHY initialization includes hierarchy construction and static-descriptor preparation, excluding compilation and path queries.

Method	W	H	R
CLEAR	19.91	67.36	111.84
2-stage PRM+A*	4.86	5.65	8.34
HOPHY	1.306	5.389	20.030
Initialization / mean query time	28	20	31

TABLE III: Path-planning comparison over 90 start–goal pairs per map. Method types: Oracle, Probabilistic, and Abstraction. Cost delta, length delta, and query time are mean±std/median relative to oracle (pixel A*).

Method	Succ.	ΔCost (%)	$\Delta\text{Len.}$ (%)	Query (ms)
Wharton (W)				
A*	90/90	—	—	2686.29 ± 2394.74 / 1996.84
RRT*	60/90	38.82 ± 17.92 / 35.32	-2.59 ± 3.16 / -2.39	3453.05 ± 2593.50 / 3044.74
RRT-C	47/90	58.21 ± 27.16 / 57.04	7.99 ± 9.33 / 7.33	232.06 ± 359.86 / 108.42
CLEAR	90/90	2.72 ± 1.57 / 2.31	-1.92 ± 4.78 / 0.00	182.66 ± 208.44 / 135.84
2-stage PRM+A*	90/90	0.00 ± 0.00 / 0.00	0.00 ± 0.00 / 0.00	655.96 ± 386.12 / 582.21
HOPHY	90/90	0.00 ± 0.00 / 0.00	0.00 ± 0.00 / 0.00	47.36 ± 93.29 / 27.95
Humphrey (H)				
A*	90/90	—	—	87726.10 ± 73544.29 / 62256.64
RRT*	72/90	33.77 ± 18.45 / 29.91	10.03 ± 16.49 / 4.62	3544.90 ± 2561.50 / 2878.57
RRT-C	61/90	38.04 ± 29.22 / 24.43	11.97 ± 23.83 / 0.33	284.58 ± 366.61 / 119.13
CLEAR	90/90	14.69 ± 3.04 / 14.49	3.98 ± 3.20 / 3.61	6502.94 ± 4093.79 / 5918.23
2-stage PRM+A*	90/90	4.14 ± 6.21 / 1.20	-3.02 ± 5.41 / -0.73	9720.27 ± 4201.28 / 9058.88
HOPHY	90/90	0.00 ± 0.00 / 0.00	0.00 ± 0.00 / 0.00	274.82 ± 116.99 / 249.88
Rainier (R)				
A*	90/90	—	—	114964.84 ± 80421.84 / 92322.37
RRT*	12/90	45.23 ± 14.85 / 42.48	2.75 ± 4.04 / 3.22	2902.68 ± 1524.33 / 2941.75
RRT-C	20/90	38.90 ± 22.59 / 29.81	2.00 ± 12.08 / -1.50	1530.38 ± 1710.82 / 621.43
CLEAR	90/90	18.60 ± 5.14 / 17.80	2.06 ± 5.35 / 2.46	8114.62 ± 3711.30 / 7772.22
2-stage PRM+A*	90/90	11.34 ± 15.31 / 6.43	-2.75 ± 7.07 / -1.26	12728.63 ± 10851.24 / 10146.66
HOPHY	90/90	0.26 ± 1.42 / 0.00	-0.05 ± 0.85 / 0.00	661.52 ± 231.07 / 629.80

best feasible attempt, while time includes all attempts. Their low Rainier success (12/90 and 20/90) makes their deltas descriptive stress-test results, and they are excluded from replanning because every changed-cost query rebuilds the tree.

B. Path-Planning Performance

We first address Q1 by comparing success, query time, and path quality over the shared 270-query benchmark. As summarized in Table III, the direct HOPHY implementation succeeds on every query and records median query times of 28, 250, and 630 ms on Wharton, Humphrey, and Rainier, respectively. These correspond to speedups of 71.4×, 249.1×, and 146.6× over oracle pixel A*, and 4.9×, 23.7×, and 12.3× over CLEAR, the fastest abstraction baseline.

Our abstraction has zero median cost and length deviation on every map and small mean deviations on Rainier, indicating close agreement with pixel A* across the evaluated queries.

Representative routes in Figure 3 show this behavior directly across the three terrain scales.

The distance-stratified view in Figure 4 shows how query time changes with route length. Pixel A* scores every expanded transition, whereas HOPHY limits computation to abstract edges requested by search and transitions inside the selected refinement corridor. Table II reports the best verified direct-hierarchy build plus static-descriptor preparation time. The HOPHY state contains 11,993, 27,666, and 51,217 GSNodes, and 134, 962, and 1025 Coarse Regions.

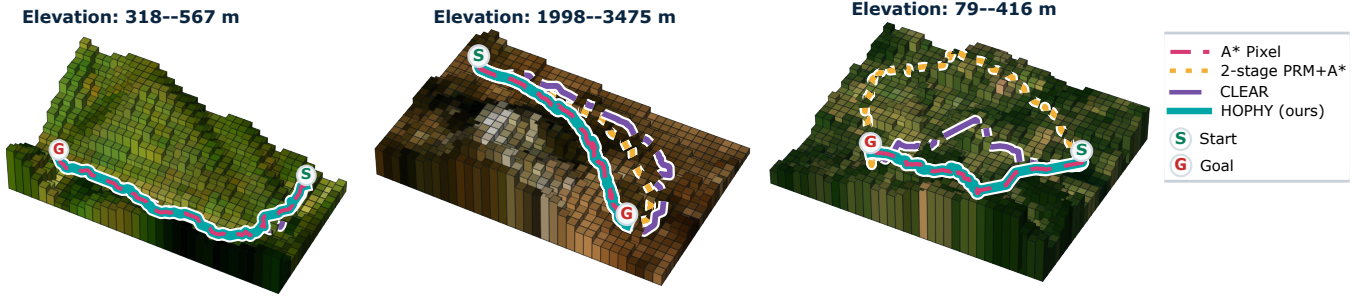


Fig. 3: Representative routes on real stepped-elevation terrain (Wharton, Humphrey, Rainier). HOPHY follows the pixel-A* corridor. CLEAR and 2-stage PRM+A* select different terrain corridors. Elevation vertically exaggerated.

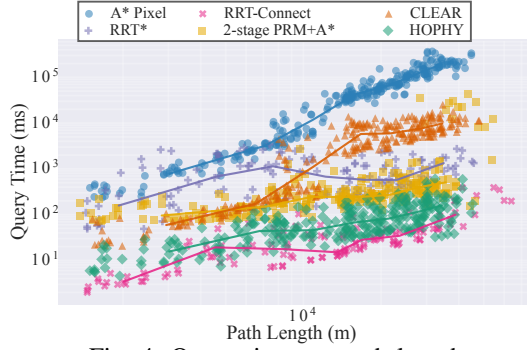


Fig. 4: Query time vs. path length

C. Replanning under Weather Change

Having established static-query performance, we address Q2: whether HOPHY retains its speed and path quality after terrain conditions change. We test both distributed cost changes, representing rain-induced loss of friction, and a localized feasibility change, representing a newly blocked region.

Setup: Using the same 90 pairs per map, light and medium rain reduce surface friction by landcover-specific fractions: 0.08/0.15 for forest, 0.10/0.20 for shrub, 0.15/0.25 for herbaceous, 0.20/0.30 for wetland, 0.12/0.22 for cropland, 0.05/0.10 for barren terrain, 0.03/0.06 for developed terrain, and 0.00/0.00 for water or snow. These are controlled sensitivity perturbations, not calibrated meteorological estimates. The blocked case instead makes one local region non-traversable. Every method receives the same modified raster and traversal objective.

The abstraction uses a weather hyperedge to identify affected GSNodes, updates their maintained state, and refines the new abstract route in its raster corridor. We compare with A* recomputed on the modified raster, 2-stage PRM+A*, and CLEAR using success, cost/length delta from A*, update time, and query time.

Together, Table IV and Figure 5 show small path-quality deviations and sub-second mean replanning under the evaluated weather and blockage changes. Our abstraction completes all queries with zero median cost and length deviation. Even on Rainier, mean update-plus-query time remains below 400 ms, compared with 89–102 s per oracle query and 15.7–38.9 s for CLEAR updates alone.

TABLE IV: Replanning under Light Rain, Medium Rain, and Blocked conditions over 90 start-goal pairs per map/condition. Metrics are mean $\pm$ standard deviation / median; cost and length deltas are relative to saved pixel-A* paths. HOPHY update time measures context-state modification and cache invalidation, excluding hierarchy construction and static-descriptor preparation. Query time includes search, required cost evaluations, and raster refinement.

Condition	Method	ΔCost (%)	$\Delta\text{Len.}$ (%)	Upd. (ms)	Query (ms)
Wharton (W) Light Rain	A*	—	—	—	4340.4 $\pm$ 5000.4/2703.5
	CLEAR	3.3 $\pm$ 2.1/2.8	0.4 $\pm$ 4.5/1.0	3425.2 $\pm$ 0.0/3425.2	232.7 $\pm$ 376.8/135.4
	2-stage PRM+A*	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	230.4 $\pm$ 398.0/118.5	230.4 $\pm$ 398.0/118.5
	HOPHY	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	0.35 $\pm$ 0.03/0.35	27.6 $\pm$ 56.3/17.4
Medium Rain	A*	—	—	—	2139.1 $\pm$ 1853.2/1544.4
	CLEAR	2.7 $\pm$ 5.2/5.5	0.8 $\pm$ 3.7/0.6	3002.4 $\pm$ 0.0/3002.4	231.8 $\pm$ 306.4/139.1
	2-stage PRM+A*	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	173.8 $\pm$ 209.4/125.6	173.8 $\pm$ 209.4/125.6
	HOPHY	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	0.34 $\pm$ 0.02/0.34	29.3 $\pm$ 59.4/19.0
Blocked	A*	—	—	—	1959.8 $\pm$ 1792.8/1473.1
	CLEAR	1.8 $\pm$ 3.1/1.8	-8.3 $\pm$ 10.6/4.8	7906.3 $\pm$ 0.0/7906.3	552.0 $\pm$ 372.8/460.1
	2-stage PRM+A*	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	1513.0 $\pm$ 1115.3/1424.1	1513.0 $\pm$ 1115.3/1424.1
	HOPHY	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	3.03 $\pm$ 2.36/2.32	32.9 $\pm$ 76.9/16.1
Humphrey (H) Light Rain	A*	—	—	—	87531.0 $\pm$ 75550.1/60220.0
	CLEAR	15.8 $\pm$ 3.4/15.4	4.0 $\pm$ 3.1/3.6	10701.5 $\pm$ 0.0/10701.5	7104.0 $\pm$ 4496.7/6113.5
	2-stage PRM+A*	3.6 $\pm$ 3.7/2.5	3.5 $\pm$ 4.9/1.4	4184.9 $\pm$ 5733.6/2020.4	4184.9 $\pm$ 5733.6/2020.4
	HOPHY	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	9.39 $\pm$ 0.43/9.27	219.7 $\pm$ 92.8/213.2
Medium Rain	A*	—	—	—	65902.1 $\pm$ 57230.8/45483.3
	CLEAR	14.8 $\pm$ 3.1/14.5	4.1 $\pm$ 3.3/3.9	11328.1 $\pm$ 0.0/11328.1	6643.1 $\pm$ 4340.1/5950.8
	2-stage PRM+A*	3.8 $\pm$ 4.0/2.7	4.1 $\pm$ 5.3/2.6	3912.7 $\pm$ 3682.9/2284.2	3912.7 $\pm$ 3682.9/2284.2
	HOPHY	0.0 $\pm$ 0.0/0.0	0.0 $\pm$ 0.0/0.0	9.21 $\pm$ 0.05/9.21	216.4 $\pm$ 91.7/195.3
Blocked	A*	—	—	—	74552.3 $\pm$ 66132.7/53651.4
	CLEAR	20.7 $\pm$ 5.3/20.6	6.3 $\pm$ 4.6/6.3	27218.2 $\pm$ 0.0/27218.2	11631.2 $\pm$ 5788.0/10492.6
	2-stage PRM+A*	3.4 $\pm$ 3.7/2.2	3.4 $\pm$ 5.3/1.3	15614.2 $\pm$ 9563.8/13588.2	15614.2 $\pm$ 9563.8/13588.2
	HOPHY	0.04 $\pm$ 0.39/0.0	-0.03 $\pm$ 0.31/0.0	4.21 $\pm$ 2.55/3.64	213.5 $\pm$ 87.6/203.0
Rainier (R) Light Rain	A*	—	—	—	101822.6 $\pm$ 61482.4/83354.8
	CLEAR	20.3 $\pm$ 4.9/20.0	1.0 $\pm$ 5.0/0.9	16125.4 $\pm$ 0.0/16125.4	8533.2 $\pm$ 4248.2/8255.5
	2-stage PRM+A*	10.7 $\pm$ 11.1/6.5	4.6 $\pm$ 7.6/1.5	14801.4 $\pm$ 40478.0/6792.7	14801.4 $\pm$ 40478.0/6792.7
	HOPHY	0.29 $\pm$ 1.55/0.0	-0.06 $\pm$ 0.81/0.0	26.98 $\pm$ 0.13/27.01	368.6 $\pm$ 173.2/319.6
Medium Rain	A*	—	—	—	88972.9 $\pm$ 56990.9/70464.0
	CLEAR	19.3 $\pm$ 4.8/19.4	2.3 $\pm$ 5.7/2.1	15744.1 $\pm$ 0.0/15744.1	8394.5 $\pm$ 4233.8/8057.1
	2-stage PRM+A*	10.2 $\pm$ 10.7/6.1	5.2 $\pm$ 7.6/2.2	14776.3 $\pm$ 38713.0/6933.3	14776.3 $\pm$ 38713.0/6933.3
	HOPHY	0.28 $\pm$ 1.53/0.0	-0.04 $\pm$ 0.72/0.0	27.25 $\pm$ 0.49/27.05	367.2 $\pm$ 169.2/318.4
Blocked	A*	—	—	—	98131.4 $\pm$ 62250.4/78354.8
	CLEAR	21.7 $\pm$ 12.8/23.2	3.8 $\pm$ 7.5/4.5	38894.3 $\pm$ 0.0/38894.3	18516.4 $\pm$ 14146.0/13445.6
	2-stage PRM+A*	9.4 $\pm$ 10.4/5.6	4.5 $\pm$ 7.2/1.8	27157.6 $\pm$ 54059.7/12815.6	27157.6 $\pm$ 54059.7/12815.6
	HOPHY	0.26 $\pm$ 1.42/0.0	-0.05 $\pm$ 0.85/0.0	7.57 $\pm$ 3.31/7.62	362.2 $\pm$ 165.8/311.3

D. Mechanism Analysis

Fig. 6 reruns both mechanism ablations on HOPHY. Enabling Coarse Region routing gives GSNode-search speedups of $1.47\times/3.25\times/1.44\times$ on W/H/R. For the same 90 affected sets and state mutations, indexed dispatch avoids a full GSNode scan and is $5.6\times/9.4\times/11.6\times$ faster.

Scaling with the affected set. We next vary the affected set from 16 to 4096 nodes on the direct 11,993-GSNode Wharton graph while fixing three membership types: terrain, agent, and weather. Their intersection identifies the changed subset. Every matched node and incident edge is then updated. Fig. 7 shows near-linear scaling, with exponents of $1.005\text{--}1.009$ and $R^2 \geq 0.9995$ across cost, feasibility, and relabel operators.

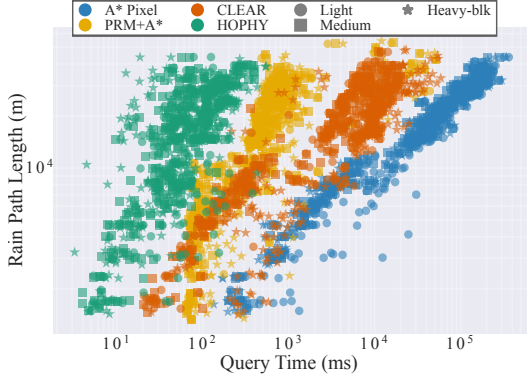


Fig. 5: Speed–quality trade-off for replanning. HOPHY path length stays close to the oracle (pixel A*) reference across rain and blockage effects.

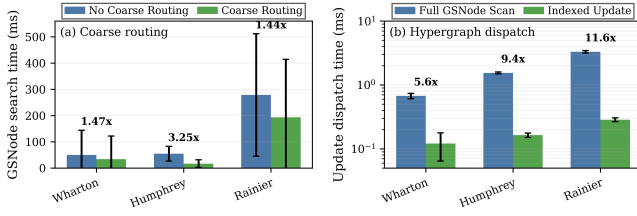


Fig. 6: HOPHY ablation (mean $\pm$ std, $n = 90$). (a) GSNode search only. (b) Update dispatch only. Both variants apply identical node and incident-adjacency mutations.

E. Multi-Agent Task Allocation

Q4 moves from individual queries to a downstream optimization that repeatedly requests path-cost estimates. On Wharton, we use HOPHY, CLEAR, 2-stage PRM+A*, and pixel A* to construct ordered task-to-task traversal-time and energy matrices for Het-MinMax-mTSP instances with 2 agents/20 tasks and 5 agents/50 tasks. An elitist genetic algorithm then minimizes makespan, $\min(\max_j \{T_{a_j}\})$. Each setting uses 10 runs with 100 iterations, population 50, elite ratio 0.01, crossover probability 0.5, and mutation probability 0.1.

As shown in Figure 8, in the 5-agent/50-task case, HOPHY reduces computation from 15677 to 198 s relative to pixel A* (79 $\times$), from 5227 to 198 s relative to 2-stage PRM+A* (26 $\times$), and from 1416 to 198 s relative to CLEAR (7.2 $\times$), with mission makespan comparable to pixel A* and 2-stage PRM+A*. Makespans are reported in seconds and computed from each method’s traversal-time matrix using the same allocation objective and GA settings. CLEAR’s abstraction underestimates travel length relative to pixel A*, yielding optimistic travel-time and makespan estimates. HOPHY’s pixel-level refinement better preserves route geometry and produces makespans close to the oracle. CLEAR’s lower estimate therefore does not establish faster mission execution.

F. Physical Jackal Feasibility Demonstration

Finally, Q5 tests whether paths generated using the proposed representation can be executed beyond the map benchmarks. We used the abstraction to generate a complete mission for a Clearpath Jackal spanning eight task locations across concrete, road, grass, and sloped muddy terrain.

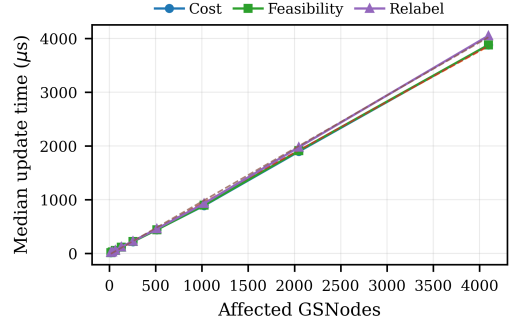


Fig. 7: HOPHY hyperedge update time vs. affected GSNodes (three membership types). Cost, feasibility, and relabel operators update the matched nodes and incident edges.

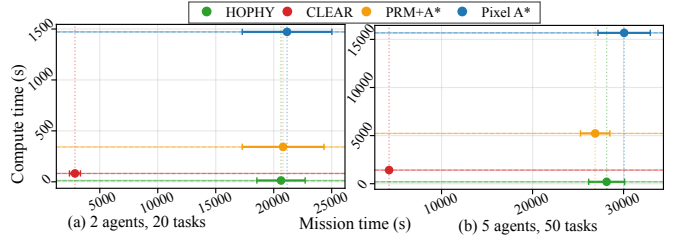


Fig. 8: Wharton MRTA results over 10 GA runs. Computation time is summed pairwise traversal-query time plus GA runtime, and mission completion time is the makespan estimated from each method’s traversal-time matrix.

Mission generation required 2.65 s, and the robot executed the resulting paths using Nav2. The aggregate feasibility outcomes are reported in Table V. Figure 9 also illustrates replanning after we simulated a construction site that blocked the planned M6 route. Updating the representation required 67.5 ms, and querying the replacement path required 137.1 ms.

Following prior outdoor-robot field reporting [18], an RTK-GNSS receiver with NTRIP corrections provided GPS waypoint localization with 5–15 cm reported horizontal error. Nav2’s Regulated Pure Pursuit controller followed the waypoints at 1.5 m/s desired speed, while an Ouster LiDAR supported local obstacle detection.

Across the paths, median map-derived slope ranges from 6.16 $^\circ$ to 20.47 $^\circ$, acceleration-based roughness RMS from 3.44 to 6.56 m/s 2 , and gyro RMS from 0.32 to 0.81 rad/s. These ranges show that the robot successfully executed paths generated by the abstraction across terrain with substantially different steepness and motion-induced disturbance levels.

VI. DISCUSSION & LIMITATIONS

Across the evaluated queries, HOPHY exhibits small path-cost deviations from pixel A*. The cost model and hierarchy parameters are calibrated for off-road UGV planning. Transfer to other domains or unseen surface types warrants separate evaluation.

When to Use: HOPHY is most beneficial when a terrain map is reused for repeated path queries, replanning, or mission optimization. Initialization takes approximately the same computation time as 28, 20, and 31 mean HOPHY

TABLE V: Physical Jackal feasibility summary. The eight-segment mission and the replanned replacement route were each executed once, results are not repeated-trial statistics.

Demonstration	Outcome	Distance (m)	Execution (s)
Eight-segment mission	8/8 completed	1505.4	994.5
Blocked-route replan	Replacement executed	251.2	164.4

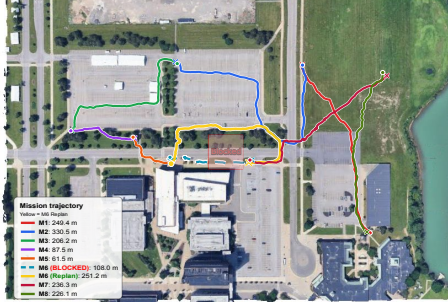


Fig. 9: Jackal mission routes and blockage-triggered replanning at the mixed-terrain outdoor site. Dashed cyan: blocked M6 plan. Yellow: 251.2m replacement. Circles and crosses mark route endpoints.

queries on Wharton, Humphrey, and Rainier, respectively.

Future Work: The maintained hypergraph backend opens several broader research directions. First, the representation is domain-agnostic in principle: applying it to urban, aerial, or multi-modal settings would require domain-appropriate segmentation and cost models, offering a path toward a unified planning backend across environments. Second, tighter integration with learning offers opportunities to replace fixed hierarchy parameters with data-driven construction and to update geometry online from onboard sensing, moving toward a self-maintaining terrain model. Third, scaling the typed update mechanism to distributed multi-agent teams—where agents hold partial views of a shared hypergraph—connects this work to decentralized planning and communication-aware mission optimization. Code, benchmark scripts, and trained model weights will be released with the paper.

VII. CONCLUSION

We presented HOPHY, a reusable hierarchical terrain representation for off-road path and mission planning. GSNodes and Coarse Regions restrict route search, while typed hyperedge intersections identify terrain regions and incident edges affected by context changes. Across the evaluated maps, our representation supports fast repeated queries and replanning without rebuilding the hierarchy, with small path-cost deviations from pixel A*. MRTA experiments demonstrate reduced planning computation, and the Jackal deployment establishes execution feasibility. These results support persistent terrain representations as a planning backend for repeated mission-level queries.

REFERENCES

[1] N. A. D. Soares and A. M. R. C. Grilo, “ARGUS: A framework for risk-aware path planning in tactical ugv operations,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.07565>

[2] R. R. Murphy, *Disaster Robotics*. MIT Press, 2014.

[3] B. P. Gerkey and M. J. Mataric, “A formal analysis and taxonomy of task allocation in multi-robot systems,” *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004. [Online]. Available: <https://doi.org/10.1177/0278364904045564>

[4] P. Poddar, E. T. Esfahani, K. Dantu, and S. Chowdhury, “Automated generation of diverse courses of actions for multi-agent operations using binary optimization and graph learning,” in *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, 2025, pp. 2901–2908.

[5] C. for Environmental Cooperation (CEC), “North american land change monitoring system (nalcms) land cover 2015, 30m,” <https://www.cec.org/nalcms/>, 2020, accessed via Google Earth Engine.

[6] N. J. P. Laboratory, “Nasadem: Nasa shuttle radar topography mission global 1 arc second v001,” <https://doi.org/10.5067/MEaSURES/NASADEM/NASADEM.HGT.001>, 2020, accessed via Google Earth Engine.

[7] P. Meshram, C. Adhivarahan, E. T. Esfahani, S. Chowdhury, C. Wang, and K. Dantu, “Clear: A semantic-geometric terrain abstraction for large-scale unstructured environments,” *IEEE Robotics and Automation Letters*, vol. 11, no. 10, pp. 11 394–11 401, 2026.

[8] A. Botea, M. Müller, and J. Schaeffer, “Near optimal hierarchical path-finding,” *J. Game Dev.*, vol. 1, no. 1, pp. 1–30, 2004.

[9] X. Zheng, M. Ma, Z. Zhong, A. Yang, L. Chen, and N. Jing, “Two-Stage Path Planning for Long-Distance Off-Road Path Planning Based on Terrain Data,” *ISPRS International Journal of Geo-Information*, vol. 13, no. 6, p. 184, May 2024. [Online]. Available: <https://www.mdpi.com/2220-9964/13/6/184>

[10] D. Lee, I. M. A. Nahrendra, M. Oh, B. Yu, and H. Myung, “TRG-planner: Traversal Risk Graph-Based Path Planning in Unstructured Environments for Safe and Efficient Navigation,” Jan. 2025, arXiv:2501.01806 [cs]. [Online]. Available: <http://arxiv.org/abs/2501.01806>

[11] S. Koenig and M. Likhachev, “Improved fast replanning for robot navigation in unknown terrain,” in *Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292)*, vol. 1. Washington, DC, USA: IEEE, 2002, pp. 968–975.

[12] J. Lim, M. Ghanei, R. C. Lawson, S. Srinivasa, and P. Tsiotras, “Lazy incremental search for efficient replanning with bounded suboptimality guarantees,” *The International Journal of Robotics Research*, vol. 43, no. 8, pp. 1175–1207, Jul. 2024.

[13] X. Meng, N. Hatch, A. Lambert, A. Li, N. Wagener, M. Schmittle, J. Lee, W. Yuan, Z. Chen, S. Deng, G. Okopal, D. Fox, B. Boots, and A. Shaban, “TerrainNet: Visual Modeling of Complex Terrain for High-speed, Off-road Navigation,” May 2023, arXiv:2303.15771 [cs]. [Online]. Available: <http://arxiv.org/abs/2303.15771>

[14] T. Fu, Z. Zhan, Z. Zhao, S. Su, X. Lin, E. T. Esfahani, K. Dantu, S. Chowdhury, and C. Wang, “AnyNav: Visual neuro-symbolic friction learning for off-road navigation,” *arXiv preprint arXiv:2501.12654*, 2025.

[15] R. Fayek and A. Wong, “Using hypergraph knowledge representation for natural terrain robot navigation and path planning,” in *Proceedings of IEEE International Conference on Robotics and Automation*, vol. 4. Minneapolis, MN, USA: IEEE, 1996, pp. 3625–3630. [Online]. Available: <http://ieeexplore.ieee.org/document/509265/>

[16] J. Motes, T. Chen, T. Bretl, M. Morales, and N. M. Amato, “Hypergraph-based Multi-Robot Task and Motion Planning,” Apr. 2023, arXiv:2210.04333 [cs]. [Online]. Available: <http://arxiv.org/abs/2210.04333>

[17] P. M. França, M. Gendreau, G. Laporte, and F. M. Müller, “The m-traveling salesman problem with minmax objective,” *Transportation Science*, vol. 29, no. 3, pp. 267–275, 1995.

[18] K. Weerakoon, A. J. Sathyamoorthy, U. Patel, and D. Manocha, “TERP: Reliable Planning in Uneven Outdoor Environments using Deep Reinforcement Learning,” Mar. 2022.

[19] P. Maul, M. Mueller, F. Enkler, E. Pigova, T. Fischer, and L. Stamatogiannakis, “BeamNG.tech Technical Paper,” BeamNG GmbH, Tech. Rep., 2021. [Online]. Available: <https://beamng.tech/blog/2021-06-21-beamng-tech-whitepaper/bng-technical-paper.pdf>

[20] P. Meshram, Y. Turkar, K. Singh, P. R. Masilamani, C. Adhivarahan, and K. Dantu, “QAL: A loss for recall-precision balance in 3d reconstruction,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Mar. 2026, pp. 7965–7974.